



3.2 数据表分类



数据表

- 在关系型数据库中，数据库表是一系列的二维数组的集合，代表了和存储数据对象之间的关系。表中的每一行称为一个记录，由若干个字段组成。字段也叫作域，表中的每一列可以称为是一个字段，字段包括了列名和数据类型这两个属性。

作者id	作者姓名	作者年龄	作者联系地址
001	张三	40	XXX省YYY市
002	李四	50	AAA省BBB市
...	...	...	...



行存表

- 行存表是默认创建表的类型，表中数据按行进行存储，即一行数据紧挨着存储，它支持完整的增删改查，适用于对数据需要经常更新的场景。

工号	姓名	性别	年龄	薪水
1001	张三	男	22	5000
1002	李四	男	23	6000
1003	王五	男	24	7000
1004	赵六	女	25	8000

1001	张三	男	22	5000	1002	李四	男	23	6000	1003	王五
------	----	---	----	------	------	----	---	----	------	------	----



创建行存表

- 语法格式

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints,  
    ...  
    columnN datatype constraints  
);
```



列存表

- 列存储表会把数据按列存放，也就是说，每一列的数据是连续存储的。这样一来，查询某一列时需要的 I/O 操作比较少，占用的存储空间也相对较少。这种存储方式适合需要做大量数据分析和统计的情况，因为它处理这些操作很高效。

工号	姓名	性别	年龄	薪水
1001	张三	男	22	5000
1002	李四	男	23	6000
1003	王五	男	24	7000
1004	赵六	女	25	8000

1001	1002	1003	1004		张三	李四	王五	赵六		男
------	------	------	------	-------	----	----	----	----	-------	---



创建列存表

- 语法格式

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints,  
    ...  
    columnN datatype constraints  
);  
WITH (ORIENTATION = COLUMN);
```

行存和列存

工号	姓名	性别	年龄	薪水
1001	张三	男	21	5000.00
1002	赵四	女	22	6000.00
1003	王五	男	30	15000.00
1004	李	女	18	3500.00

按照数据的存储方式，表分为两种

- 行存储表
- 列存储表

数据存储序列方向 ↓	行存储 (记录顺序从上到下)				
	Row1	1001	张三	男	21
	Row2	1002	赵四	女	22
	Row3	1003	王五	男	30
	Row4	1004	李	女	18
数据存储序列方向 ↓	列存储 (记录顺序从上到下)				
	C1	1001	1002	1003	1004
	C2	张三	赵四	王五	李
	C3	男	女	男	女
	C4	21	22	30	18
	C5	5000.00	6000.00	15000.00	3500.00

行存储

列存储

写入过程要把记录拆开后不同的列数据分别写入不同的存储区域，多次写入过程会导致IO次数增加，效率相对较慢。

select name,age from employee;
对于列存储表的查询，只要扫描少量所需要列对应的存储即可，IO开销比较小。

行列存储比较

- 行存表

- 表的数据将以行式存储，每行的所有属性存储到一起。
- 适合查询一行的所有属性。
- INSERT、UPDATE效率高。
- 默认方式。

- 列存表

- 表的数据将以列式存储，每列的多个纪录存储到一起。
- 适用于海量数据查询，减少磁盘访问数据量。投影高效。
- INSERT、UPDATE效率略低。



分区表

- 分区表是把逻辑上的一张表根据某种方案分成几张物理块进行存储。这张逻辑上的表称之为分区表，物理块称之为分区。分区表是一张逻辑表，不存储数据，数据实际是存储在分区上的。
- 对于大多数用户使用场景，分区表和普通表相比具有以下优点：
 - 改善查询性能：对分区对象的查询可以仅搜索自己关心的分区，提高检索效率。
 - 增强可用性：如果分区表的某个分区出现故障，表在其他分区的数据仍然可用。
 - 方便维护：如果分区表的某个分区出现故障，需要修复数据，只修复该分区即可。
 - 均衡I/O：可以把不同的分区映射到不同的磁盘以平衡I/O，改善整个系统性能。



分区表

- openGauss数据库支持范围分区表、列表分区表、哈希分区表：
 - 范围分区表：将数据基于范围映射到每一个分区，这个范围是由创建分区表时指定的分区键决定的。这种分区方式是最为常用的。
 - 列表分区表：将数据基于各个分区内包含的键值映射到每一个分区，分区包含的键值在创建分区时指定。
 - 哈希分区表：将数据通过哈希映射到每一个分区，每一个分区中存储了具有相同哈希值的记录。

分区表基本操作 (1)

- 创建示例

- 创建范围分区表student1，含有三个分区，分区键为整数类型。三个分区的范围分别为：score1 < 60, 60 <= score2 < 80, 80 <= score3 < 90。

```
CREATE TABLE student1
(
    name varchar(30),
    score int,
    birthday timestamp
)
PARTITION BY RANGE(score)
(
    PARTITION score1 VALUES LESS THAN(60),
    PARTITION score2 VALUES LESS THAN(80),
    PARTITION score3 VALUES LESS THAN(90)
);
```

- 删除示例

- 删除分区score1

```
ALTER TABLE student1 DROP PARTITION score1;
```

分区表基本操作 (2)

- 增加示例

- 增加分区score4, 范围为 $90 \leq \text{score4} \leq 100$

```
ALTER TABLE student1 ADD PARTITION score4 VALUES LESS THAN (101);
```

- 重命名示例

- 重命名分区score2为score2_new:

```
ALTER TABLE student1 RENAME PARTITION score2 TO score2_new;
```

- 查询示例

- 查询分区score2

```
SELECT * FROM student1 PARTITION (score2_new);
```

分区表基本操作 (3)

- 分区表索引

- 创建分区表索引 student1_index1, 不指定索引分区的名字

```
CREATE INDEX student1_index1 ON student1 (score) LOCAL;
```

- 创建分区索引 student1_index2, 并指定索引分区的名字

```
CREATE INDEX student1_index2 ON student1 (score) LOCAL  
(  
    PARTITION score1_index,  
    PARTITION score2_index TABLESPACE ts2,  
    PARTITION score3_index TABLESPACE ts3  
) TABLESPACE ts0;
```

```
ALTER INDEX student1_index2 RENAME PARTITION score1_index TO score1_index_new;
```

分区设计总结

- 针对已经存在的表进行分区，最好将数据迁移完后再建索引。
- 如果要充分使用分区表的查询优势，必须使用分区时的字段作为过滤条件。
- 需要注意分区后就没有全局唯一性了，各个分区之间是可以有重复的uuid。
- 对于分区键条件查询，效率非常高。
- 分区的字段必须是非空，类似于案件的立案日期和结案日期就不能用作分区字段。
- 分区的备份可以单独备份各个分区，但是如果备份所有分区只能备份整个schema。